Instance Segmentation

Instance Segmentation
1. Course Content
2. Case Demonstration
3. Source Code Analysis

1. Course Content

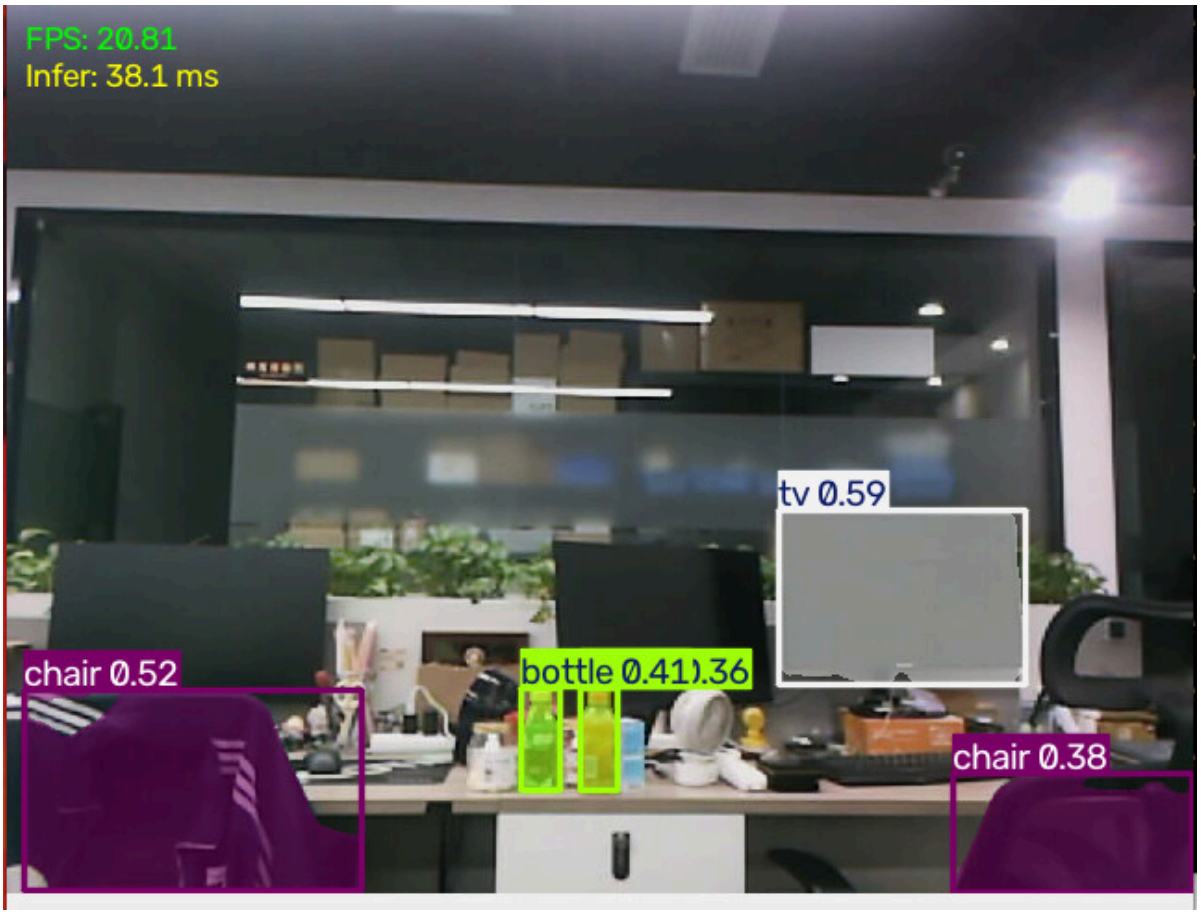
- Use the yolo instance segmentation model to detect and segment the target objects in the camera image.

2. Case Demonstration

```
ros2 launch yahboomcar_bringup camera_demo.launch.py demo:=yolo_seg
```

The detection image consists of the following elements:

- **Semi-transparent colored segmentation mask**: The model generates a pixel-level segmentation mask for each detected target, overlaid on the original image as semi-transparent regions of different colors, precisely outlining the contour boundaries of the target.
- **Bounding Box**: The rectangular box around each target, marking the detection range of the target.
- **Class name and confidence**: Above the bounding box, the target class name (such as person, car, etc.) and the confidence predicted by the model (0~1) are displayed.
- **Upper-left status bar**: Green text displays the real-time frame rate FPS, and yellow text displays the single-frame YOLO inference time (milliseconds).



3. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/yolo/yolo_seg.py
```

Constructor description:

- `declare_parameter('init_pose', [90, 180, 0, 0, 90, 30])`: Declares the robot arm initial pose parameter, 6 joint angle values (degrees), which are arm1~arm5 and the gripper angle in order.
- `declare_parameter('model_path', '')`: Declares the YOLO model file path parameter, defaulting to empty; the model file must be specified via the `-p` command line.
- `declare_parameter('max_det', 5)`: Declares the maximum number of detected objects per frame, defaulting to detecting at most 5 objects.
- `create_client(ArmPosition, 'set_arm_joints')`: Creates a robot arm joint control service client, used to move the robot arm to the initial pose.
- `CvBridge()`: Creates a bridge for conversion between ROS image messages and OpenCV images.
- `create_subscription(Image, 'camera_rgb_topic', ...)`: Subscribes to the camera RGB image topic; the callback is triggered each time an image frame is received.

- `YOLO(self.model_path, task='segment')`: Loads the YOLO model, with the task type being `segment` (instance segmentation); compared with the `detect` mode, it additionally outputs a segmentation mask.
- `_init_arm_pose()`: Moves the robot arm to the initial pose after the model is loaded.

_init_arm_pose function description:

- `wait_for_service(timeout_sec=10.0)`: Waits for the robot arm joint control service to come online; if it times out after 10 seconds, the initial pose setup is skipped.
- `ArmPosition.Request()`: Constructs a joint control request, where `request_joints` is the initial pose angle list.
- `call_async(req)`: Sends the joint control request asynchronously.
- `add_done_callback(self._init_arm_done_callback)`: Registers a callback function to check the execution result after the request completes.

get_RGBImageCallBack function description:

- `imgmsg_to_cv2(rgb_msg, desired_encoding='bgr8')`: Converts the ROS Image message into an OpenCV BGR format image.
- `self.yolo_model.predict(rgb_image, stream=True, max_det=self.max_det, verbose=False)`: Performs YOLO instance segmentation inference on the current frame; `stream=True` enables streaming output to save memory.
- `res.plot()`: Draws the detection results (bounding boxes, segmentation masks, class names, confidence) onto the image; the segmentation masks are overlaid on the target pixels as semi-transparent colored regions.
- FPS calculation: Counts the number of frames within 1 second to calculate the real-time frame rate.
- `cv2.putText(...)`: Overlays and displays the FPS and the single-frame inference time (milliseconds) in the upper left corner of the image.
- `cv2.imshow(...)` / `cv2.waitKey(1)`: Displays the segmentation detection image; `waitKey(1)` refreshes the window.

```
class YOLO_SEG(Node):
    def __init__(self,name):
        super().__init__(name)
        self.declare_parameter('init_pose', [90, 180, 0, 0, 90, 30])
        self.init_pose =
self.get_parameter('init_pose').get_parameter_value().integer_array_value
        self.declare_parameter('model_path','')
        self.model_path =
self.get_parameter('model_path').get_parameter_value().string_value
        self.declare_parameter('max_det', 5)
        self.max_det =
self.get_parameter('max_det').get_parameter_value().integer_value
        self.set_arm_client = self.create_client(ArmPosition, 'set_arm_joints')
        self.rgb_bridge = CvBridge()
        self.msg2img_bridge = CvBridge()
        self.count=0
        self.sub_rgb =
self.create_subscription(Image,'camera_rgb_topic',self.get_RGBImageCallBack,1)
```

```
        self.frame_count=0
        self.prev_time = time.time()
        self.frame_count = 0
        self.fps = 0.0

        self.yolo_model = YOLO(self.model_path, task='segment')
        self.get_logger().info(Fore.GREEN + 'YOLOV26-Seg Model Loaded.' +
Fore.RESET)
        self._init_arm_pose()

    def _init_arm_pose(self):
        if not self.set_arm_client.wait_for_service(timeout_sec=10.0):
            self.get_logger().warn("Failed to connect to set_arm_joints server,
skipping init arm pose")
            return
        req = ArmPosition.Request()
        req.request_joints = list(self.init_pose)
        future = self.set_arm_client.call_async(req)
        future.add_done_callback(self._init_arm_done_callback)


    def _init_arm_done_callback(self, future):
        result = future.result()
        if not result.success:
            self.get_logger().warn(f"Init arm pose failed:
{list(result.result_joints)}, msg: {result.message}")


    def get_RGBImageCallBack(self,rgb_msg):
        rgb_image = self.rgb_bridge.imgmsg_to_cv2(rgb_msg,
desired_encoding='bgr8')
        infer_start = time.time()
        results = self.yolo_model.predict(rgb_image, stream=True,
max_det=self.max_det, verbose=False)
        for res in results:
            if res.boxes is not None:
                annotated_frame = res.plot()
        infer_end = time.time()
        self.frame_count += 1
        curr_time = time.time()
        time_diff = curr_time - self.prev_time
        if time_diff >= 1.0:
            self.fps = self.frame_count / time_diff
            self.frame_count = 0
            self.prev_time = curr_time
        cv2.putText(annotated_frame,f"FPS: {self.fps:.2f}",(10,
25),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),1,)
        infer_time = (infer_end - infer_start) * 1000
        cv2.putText(annotated_frame, f"Infer: {infer_time:.1f} ms", (10,
45),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)

        cv2.imshow("YOLO26-Seg Panel", annotated_frame)
        cv2.waitKey(1)
```